Project Proposal
GitHub Security Gate

Team Implementation Reference

Aditya Chatterjee
Arvind Dharmesh Parekh
Chengyuan Sun

Project Goal and Scope

Original must-have deliverable

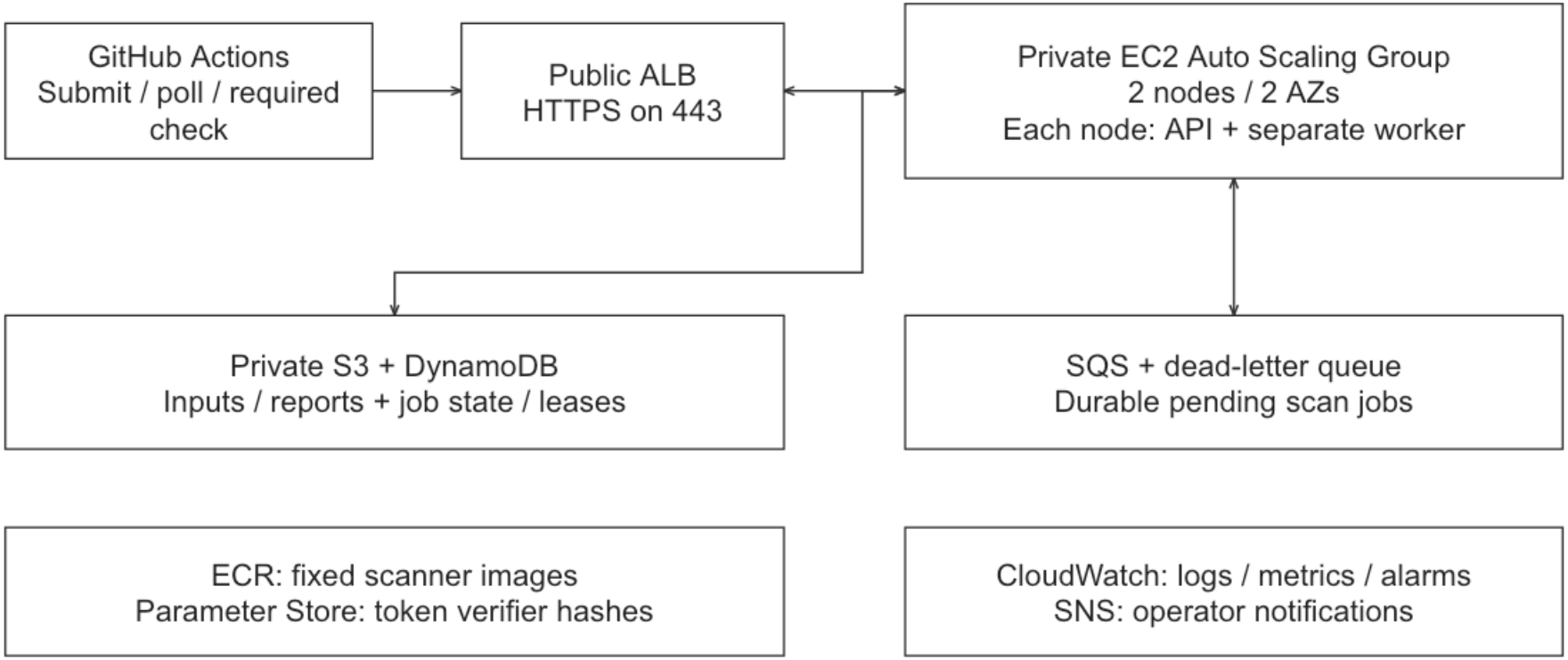
- One team repository. Supplied JavaScript SAST and Node / Express backend.
- Required GitHub check: block HIGH findings and allow a corrected commit.
- Private redacted reports with expiring links, plus bounded Docker scans.

Nice to have (not required)

- Pentesting, more languages, and multiple teams.
- Custom dashboard, chat notifications, and rule-management UI.
- Automatic capacity scaling beyond two hosts and scheduled rescans.

Goal and final demo are unchanged. Deployment changes address the professor's feedback. Nice-to-haves are future options, not commitments for this project.

Architecture Diagram



Each worker receives text and launches one resource-limited scan container.
Supporting services on the bottom row serve both EC2 nodes.

Data Flow

1. **GitHub submits the exact PR commit**
   Authenticated POST /scan-jobs includes bounded files and an idempotency key.
2. **API persists the input and queues the job**
   S3 holds temporary source. DynamoDB tracks the job. SQS accepts its job ID.
3. **API returns HTTP 202 with a scan ID**
   GitHub shows Queued and polls status. Busy workers do not cause HTTP 429.
4. **A worker claims the job and runs the sandbox**
   One bounded container scans text. The supervisor enforces its deadline.
5. **Supervisor saves a redacted result**
   Write S3 report, commit DynamoDB result, then acknowledge the queue message.
6. **Workflow completes the required check**
   Show findings / report link. Bind the outcome to the exact commit SHA.

AWS Services and Responsibilities

Request and compute

- ALB terminates HTTPS and routes only to healthy API targets.
- Auto Scaling maintains two private EC2 nodes. Each runs an API and worker supervisor.
- ECR stores the tested, digest-pinned scanner image.

State and operations

- SQS absorbs bursts. A dead-letter queue holds repeated processing failures.
- DynamoDB stores status, idempotency records, and worker leases. S3 stores artifacts.
- Parameter Store holds verifier configuration. CloudWatch and SNS expose failures.

Terraform defines these resources. Two AZs use separate outbound paths.
One codebase contains distinct API, supervisor, and scanner components.

Original Demo and Supporting Checks

Unchanged final project demo

- Open a JavaScript PR with a HIGH issue. Its required check fails.
- Push corrected code. The new commit passes after a complete scan.
- Open the saved, redacted S3 report through an expiring link.
- Demonstrate that an incomplete scan cannot pass.

Feedback-driven verification

- Verify failover and queue handling when developers push together.
- Verify endpoint authentication, token rotation, and shared Terraform state.
- Retain sandbox limits and test the existing scanner / report behavior.

Same deliverable: A GitHub SAST security gate running in AWS Academy Learner Lab.
Supporting checks validate deployment changes. They do not add new product features.

Data Storage and Retention

Application data

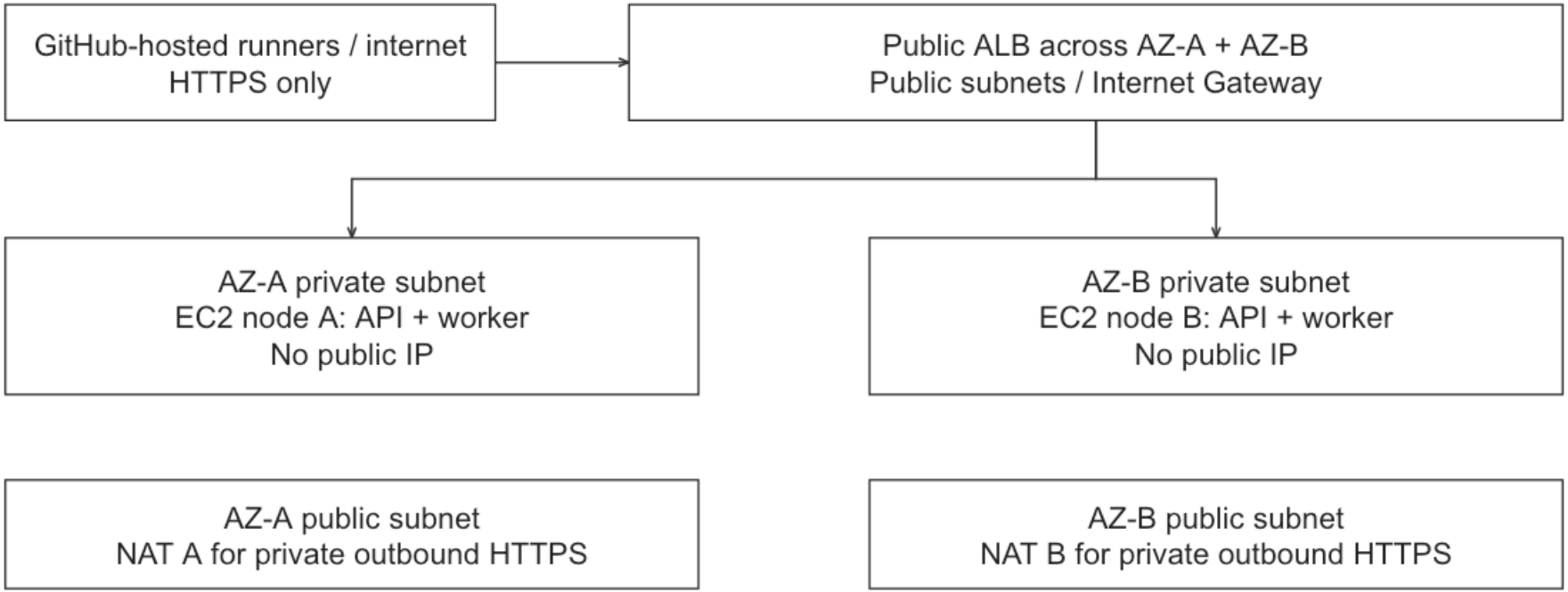
- S3 input bucket: encrypted source manifest for queued scans. Delete after a terminal result.
- S3 report bucket: sanitized findings and immutable attempt reports for 30 days.
- DynamoDB: job status, SHA, versions, lease owner, report pointer, and expiry.

Recovery and privacy

- Abandoned source expires by a 1-day lifecycle rule. Deletion is asynchronous.
- Report versioning supports overwrite recovery. Noncurrent versions expire after 7 days.
- Logs expire after 14 days. State bucket has separate retention and access rules.

This replaces the earlier “no source persistence” claim: queued scans need temporary inputs. Never persist raw secret evidence, bearer tokens, or credentials in reports or logs.

VPC and Network Boundaries



S3 / DynamoDB gateway endpoints serve the private route tables.
Security groups permit ALB-to-API traffic only. Workers expose no inbound port.

Security Group and API Rules

Boundary	Allowed traffic / control
Internet to ALB	TCP 443 from 0.0.0.0/0. No fixed GitHub runner IP list.
ALB to EC2 API	TCP 3000 from the ALB security group only.
Internet to EC2 / worker	No public IP, SSH, 443 listener, or published Docker port.
API request authorization	Repo-scoped bearer token, input validation, and per-node rate limits.
API to container launcher	No direct access. A separate supervisor owns the fixed launcher.

HTTPS encrypts traffic. It does not identify the caller. The token is the identity control.
A stolen token remains a risk, so rotation, redaction, and bounded work are required.

Availability and Deployment

Baseline capacity

- ASG min / desired / max = 2 / 2 / 2, distributed across two AZs.
- ALB removes unhealthy API targets. ASG replaces failed EC2 nodes.
- Each node processes one job at a time. SQS holds additional jobs.

Update and failure behavior

- State lives in S3 / DynamoDB / SQS, so another node can resume work.
- Use a launch template and one-at-a-time instance refresh after a smoke test.
- Pin image digests and keep the previous launch template version for rollback.

Two hosts remove the original single-instance dependency. Lab-wide shutdowns still interrupt work. Dynamic scaling beyond two nodes is optional after the queue and lease model works.

False Positives and Policy

Baseline exception handling

- Keep one exact, reviewed allowlist in the trusted scanner configuration.
- Each entry identifies file, rule, finding fingerprint, reason, and expiry.
- Use the protected policy version. PR content cannot approve its own exception.

Scope limits

- Record suppressed findings separately in the sanitized report.
- Unapproved HIGH findings remain blocking, even if an inline ignore comment exists.
- A rule-management UI and automatic approval workflow are nice-to-haves.

Passing means a complete scan below the configured blocking threshold.
Regex matching can still miss vulnerabilities or flag safe code.

Learner Lab Preflight

Verify before infrastructure work

- Two permitted EC2 instances, two AZs, ALB, ASG, NAT, SQS, DynamoDB, and S3.
- ECR, Parameter Store, CloudWatch, and the allowed instance profile / role.
- A DNS name we control plus a validated ACM certificate for the ALB.

Agree before application work

- Confirm lab session shutdown / restart behavior and service quotas.
- Estimate active-session spend for both NAT gateways, ALB, EC2, and storage.
- Confirm required GitHub checks are supported and protect workflow changes.

Owner: Aditya. Complete feasibility and setup before handing implementation tasks over.
If a gate fails, resolve the design first so neither teammate has to discover setup blockers.

Cost Controls and Tradeoffs

Cost decisions

- Use two small, lab-permitted instances during development and demos.
- Keep fixed capacity at two. Additional capacity and automatic scaling wait.
- Expire source, reports, and logs. Tag resources by project and owner.

Availability decisions

- Two NAT gateways avoid one shared outbound failure point.
- ALB and shared durable services cost more than the former single-host design.
- Stop the project deliberately after sessions, accepting unavailable checks then.

Terraform supports a documented stop / resume procedure and teardown.
Retain the state bucket and agreed reports. Stopping EC2 alone does not stop ALB / NAT charges.

Team Ownership and Handoffs

Proposed lead	Owns	Handoff / completion
Aditya Chatterjee	All setup, AWS / Terraform, API, contracts, integration	Working baseline, dummy worker, setup guide and examples.
Arvind Dharmesh Parekh	GitHub workflow, polling, retries, required check, end-to-end tests	PR workflow shows correct pass / fail and report links.
Chengyuan Sun	Real scanner worker, Docker limits, recovery, worker tests	Worker matches agreed jobs / results and handles failures.

Order: Aditya builds the foundation first. Arvind and Chengyuan then work in parallel.
All three test and demonstrate the same original SAST project. Proposed split for the group.

Same Workflow, Clearer Status

What GitHub shows

- Submit once and receive a job ID with Queued status. Poll every 5 seconds.
- Show Running after a worker acquires its lease. Show counts and report link at completion.
- Show Infrastructure error on timeout / failure, with a rerun instruction.

Public report access

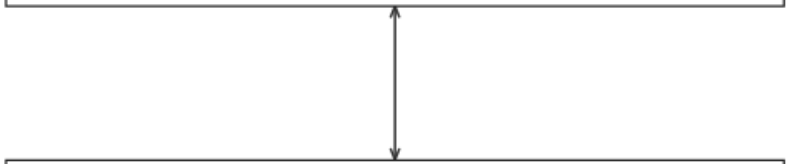
- S3 stays private. Authenticated link requests return a 15-minute presigned URL.
- Anyone holding that URL can read the redacted report until it expires.
- A manual GitHub workflow refreshes a link for an existing job without rescanning.

Worker capacity no longer causes a busy HTTP 429. Queue wait is visible.
Only complete results for the current commit can satisfy its required check.

Scan Isolation and Privilege Split

Public-facing Express API
No Docker socket or launcher access

Separate worker supervisor
SQS claim / source fetch / persistence



Fixed Docker scan image
All SAST rules / no network / no credentials

Shared durable job state connects the API and worker.
The API never launches arbitrary containers.

- One sandbox per scan, one active scan per node.
- Proposed limits: 0.5 vCPU, 256 MiB, 10 seconds.
- Supervisor kills failed jobs and removes containers.

Acceptance Cases: Scan and Access

ID	Input / condition	Expected outcome
01	Vulnerable course fixture	Complete report, HIGH outcome fails the check.
02	Corrected fixture	Complete report with no blocking findings passes.
03	MEDIUM-only findings	Pass with visible warnings under HIGH threshold.
04	Approved exception	Suppress the exact finding and show its audit record.
05	Invalid / unapproved exception	Keep HIGH blocking. A PR cannot approve its policy.
06	Expired exception	Keep the finding active and show the expiry.
07	No eligible JavaScript	Return an explicit complete zero-file report.
08	Oversized or malformed body	Reject with 400 / 413 before queuing work.
09	Missing token / wrong repository	Return 401 / 403. No new job or report link.
10	Sandbox deadline / memory limit	Kill the attempt, record error, keep the API healthy.

Core cases 1-10. Store evidence by job ID, exact SHA, and scanner / policy version.

Acceptance Cases: Reliability

ID	Input / condition	Expected outcome
11	API node fails	ALB uses survivor. ASG replaces the failed host.
12	Worker node dies mid-scan	Job reappears after visibility/lease expiry and resumes.
13	Duplicate message / POST retry	One canonical result. Duplicate delivery is harmless.
14	Several PRs arrive together	202 Queued for admitted jobs, then terminal results.
15	Same PR is pushed repeatedly	Cancel old workflow. Only the newest SHA can pass.
16	429 / 503 / transport failure	Use the bounded retry policy, then show an error.
17	S3 or DynamoDB commit fails	No passing result. Retry safely or record failure.
18	Expired report URL	Old link fails. Authorized refresh issues a new link.
19	Service token rotates	New token works on both nodes. Revoke the old token.
20	Concurrent Terraform apply	Shared state lock permits one writer. No state in Git.

Core cases 11-20. Record actual recovery time and remaining lab limitations.

API Contract for Implementation

Route	Contract / response
POST /scan-jobs	Bounded files + repo / PR / SHA. 202 after durable queue acceptance.
GET /scan-jobs/{jobId}	Repo-authorized state and sanitized summary. Poll every 5 seconds.
POST /scan-jobs/{jobId}/report-link	Repo-authorized link refresh. 15-minute signed S3 GET URL.
GET /health and /ready	Minimal health data. ALB uses /ready on private API port 3000.
Errors	400 / 413 input, 401 / 403 auth, 409 key conflict, 429 / 503 retryable.

Common statuses: SUBMITTING, QUEUED, RUNNING, COMPLETE, ERROR, TIMED_OUT.
Only COMPLETE plus an outcome bound to the requested SHA can finish the gate.

Queue, Leases, and Idempotency

Durable submission

- Persist source and a DynamoDB job/outbox record before sending its SQS message.
- Return 202 only after SQS accepts the message. Retry unfinished enqueue work.
- Repeated POSTs and SQS deliveries reuse the same stable job identity.

Safe processing

- Claim RUNNING with an atomic lease. Only its current owner can commit the result.
- Start with 90-second queue visibility / lease and renew during processing.
- Write report, commit its pointer, then delete the queue message.

A crashed worker loses its lease and the unacknowledged job becomes available again.
Do not promise exactly-once execution. Make duplicate work and commits safe.

Workflow Retries and Queueing

Situation	Developer sees / workflow action
Workers are occupied	202 + job ID. Show Queued, then Running. No capacity-busy retries.
API rate limit / transient outage	At most 3 retries: 2s, 5s, 10s + jitter. Same idempotency key.
Job still not terminal at 5 minutes	Fail as infrastructure timeout. Keep the job ID and rerun instruction.
Another push to the same PR	Cancel older workflow. Require the new commit's own result.
Several different PRs	Each has its own concurrency group and queued job.

Before host 2: Shared job state, atomic leases, idempotency, queue recovery, and burst tests.
Adding an instance alone cannot fix lost jobs or incorrect check results.

Service Token Storage and Rotation

Storage and scope

- GitHub Actions secret SAST_SERVICE_TOKEN holds the client bearer token.
- Parameter Store SecureString holds current / previous token hashes and allowed repo ID.
- Each API refreshes this verifier configuration every 60 seconds. No token in images.

Rotation procedure

- Generate a new random token and add its verifier while retaining the old hash.
- After both nodes refresh, update the GitHub secret and run a probe on each node.
- Keep a 10-minute overlap, then remove the old hash and verify old-token rejection.

Rotate after exposure or a member change, and before the final demo. Aditya owns the runbook.
Never put bearer values in Terraform variables, state, user-data, reports, or logs.

Terraform and Remote State

What Terraform owns

- VPC, subnets, routes, NAT, ALB, launch template, ASG, security groups.
- S3, SQS / DLQ, DynamoDB, ECR, logs, alarms, and configuration references.
- Inputs reference permitted lab roles, certificate ARN, image digest, and environment.

Where state lives

- Separate private S3 state bucket. Key: cs6620/dev/terraform.tfstate.
- Enable encryption, versioning, Block Public Access, and native S3 lockfiles.
- One shared backend for all three members. Never commit state or live secrets.

Use use_lockfile = true with a pinned Terraform version that supports it.
Keep the dependency lockfile in Git. The state bucket has no report-download route.

Foundation and Handoff Package

Stage / owner	Concrete handoff or result
Aditya: feasibility + setup	Confirm lab, HTTPS and budget. Create repo, Terraform state and AWS baseline.
Aditya: working API	Publish contracts and a runnable submit/status/report flow with a dummy worker.
Handoff to Arvind	API URL, secret setup instructions, example responses, workflow acceptance tests.
Handoff to Chengyuan	Queue/job schemas, sample jobs, source/result examples, limits and test commands.
Aditya leads integration	Combine finished components. Each owner fixes her/his part. Test and demo together.

The handoff must run before parallel implementation starts. Examples and mocks remove guesswork. Keep setup, schemas, test commands and runbooks in the same repository. Never document live secrets.

Setup First, Then Parallel Work

Gate	Evidence needed to proceed	Lead
A. Feasibility + setup	Lab, roles, budget, HTTPS, repository and shared state verified.	Aditya
B. Working foundation	API, contracts, AWS baseline and dummy job work. Handoff guide is ready.	Aditya
C. Parallel build	GitHub workflow and real worker develop against the agreed baseline.	Arvind + Chengyuan
D. Integration	Combine components and run PR, burst, resource-limit and failover tests.	Aditya + team
E. Release readiness	20-case evidence, rotation drill, rollback notes and original final demo.	All three

Aditya owns every prerequisite. Arvind and Chengyuan have independent paths after the handoff. The project goals, final demo and two-node baseline remain unchanged.

Feedback Decisions and Open Gates

Feedback	Selected decision / reference
Scope and single-host failure	Same SAST gate and final demo. ALB / two nodes improve deployment reliability. Slides 2-3, 6, 10.
Public 443 and endpoint exposure	443 exists only on ALB. Token, validation and rate limits protect API. Slide 9.
Busy scans and rapid pushes	Durable queue, exact retry budget, per-PR cancellation and SHA binding. Slides 20-21.
Token storage and rotation	GitHub secret + SSM verifier hashes, timed overlap and revocation. Slide 22.
IaC and team ownership	Shared encrypted/locked S3 state, proposed named owners, staged handoffs. Slides 14, 23-25.

Before build: Confirm lab permissions, certificate, cost, and proposed responsibilities.
Then work from the single architecture, contracts, and acceptance criteria in this deck.